

Implementation and Analysis of a Victim Cache in SimpleScalar

Team:

Peta Shanmuga Teja (25CS06007)

Khem Singh (25CS06004)

Group:6

1. Objective

Integrate a victim cache between the Level-1 (L1) and Level-2 (L2) data caches in SimpleScalar (`sim-cache.c`) and analyze its impact on memory performance versus a baseline without a victim cache.

2. Introduction

A victim cache (VC) is a small, fully-associative buffer that temporarily holds lines evicted from L1. On an L1 miss, the VC is probed before consulting L2. If the block is found (victim hit), the access is satisfied at VC latency and the block is swapped back into L1, converting many conflict misses into hits with minimal hardware cost. This report implements that idea in SimpleScalar and quantifies its benefits across several workloads.

3. Implementation Details

All changes are concentrated in `sim-cache.c` of SimpleScalar-3.0. We present the *diff-level* changes by comparing the **baseline** and **VC-enabled** versions.

3.1 New data and configuration

- New global pointer: `static struct cache_t *victim_cache = NULL;`

- **Config knob:** `static int victim_entries;` — number of VC lines.
- **Command-line option:** `-victim:entries <n>`
 - `0` disables the victim cache (baseline behavior).
 - `n > 0` enables a fully-associative VC with `n` entries.

3.2 Construction of the victim cache

On simulator init (after DL1/DL2 creation), create the VC when `victim_entries > 0`:

- Name format: `victim_%d` (avoids issues with `:` in derived stat names).
- Organization: **1 set**, associativity = `victim_entries` (fully-associative).
- Block size: matches DL1 block size.
- **Hit latency:** 1 cycle (kept small to reflect a tiny on-chip buffer).

3.3 Access path changes

- **L1 miss handling** (e.g., inside `dl1_access_fn` or the L1 access path):
 1. On an L1 miss and `victim_entries > 0`, **probe VC**.
 2. **VC hit:** satisfy access at VC latency and **swap** the VC line with the would-be L1 victim way to keep L1 hot. Update LRU/dirty bits accordingly.
 3. **VC miss:** proceed to L2 as in the baseline.
- **L1 evictions:** the evicted L1 block is inserted into the VC (possibly evicting a VC line). If a VC line is dirty on eviction, perform a write-back to the next level.

3.4 Latency adjustments

To model a realistic hierarchy, DL2/IL2 **hit latencies** are raised from 1 to **6 cycles**. VC remains at **1 cycle**.

3.5 Statistics and reporting

Register VC statistics alongside existing cache stats:

- `vc.hits`, `vc.misses`, `vc.evictions`, `vc.writebacks`.
- Ensure `cache_reg_stats(victim_cache, sdb);` is invoked when VC is enabled.

3.6 Build and run interface

No Makefile changes are required beyond recompiling after edits. Run examples:

```
./sim-cache -victim:entries 0 -cache:dl1 dl1:128:32:1:l -cache:dl2 ul2:1024:64:4:l  
../Labs/Pisa/anagram  
./sim-cache -victim:entries 4 -cache:dl1 dl1:128:32:1:l -cache:dl2 ul2:1024:64:4:l  
../benchmarks/go.alpha 50 9 <../benchmarks/2stone9.in>
```

4. Effective Miss Rate Definition

Let total L1 accesses = A , L1 misses = M_{dl1} , VC probes = P_{vc} (on L1 misses), and VC misses = M_{vc} .

- **VC disabled ($victim_entries = 0$):**
 $EffectiveMissRate = M_{dl1} / A$
 - **VC enabled ($victim_entries > 0$):**
 $EffectiveMissRate = M_{vc} / A$
(Because a VC hit upgrades an L1 miss into a hit at VC; only VC misses flow to L2.)
-

5. Observations and Results

We compare the **baseline** file (no VC) vs the **VC-enabled** file using identical cache parameters and inputs. Reported figures are per-benchmark.

5.1 Miss-rate and effective miss-rate

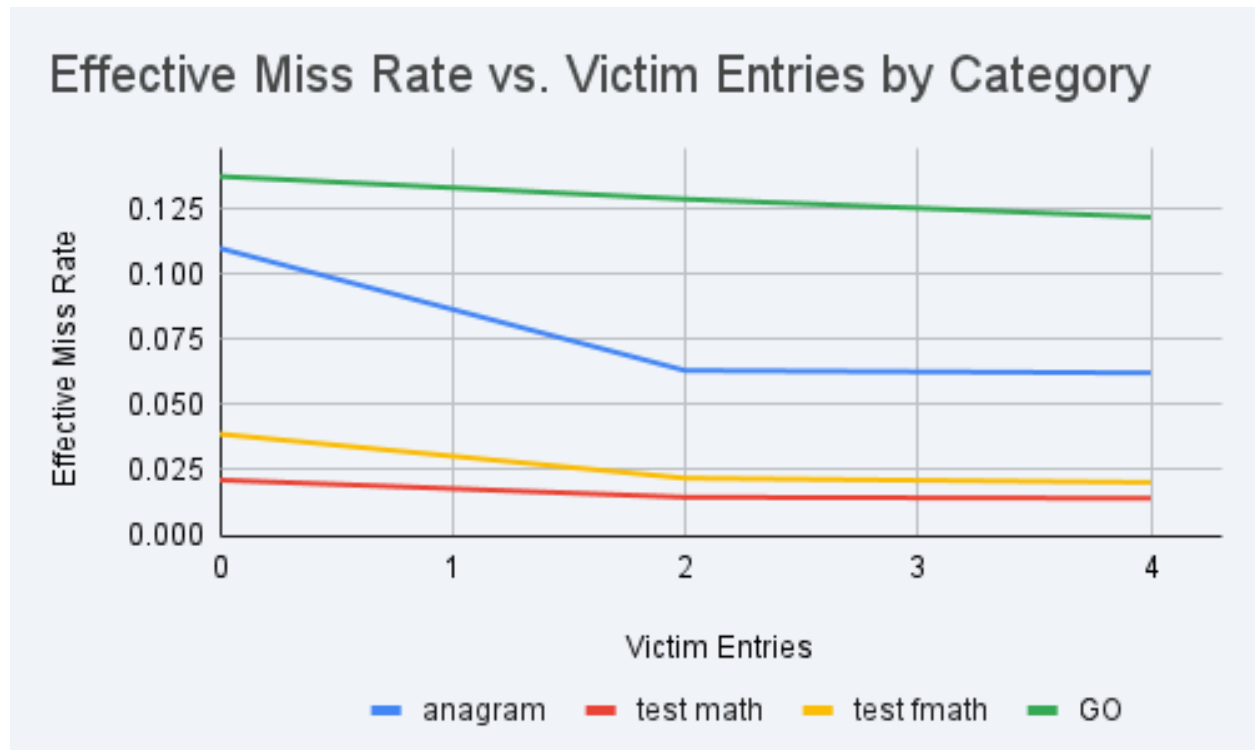
Benchmark	victim_entries	L1 Miss Rate	VC Miss Rate	Effective Miss Rate
anagram	0	0.1098	—	0.1098
anagram	2	0.1098	0.3375	0.0630
anagram	4	0.1098	0.3325	0.0621
test-math	0	0.0211	—	0.0211
test-math	2	0.0211	0.4254	0.0145
test-math	4	0.0211	0.4147	0.0141
test-fmath	0	0.0387	—	0.0387
test-fmath	2	0.0387	0.3393	0.0218
test-fmath	4	0.0387	0.3140	0.0202
go.alpha	0	0.1373	—	0.1373
go.alpha	2	0.1373	0.6876	0.1286
go.alpha	4	0.1373	0.6506	0.1217

Takeaways

- Across conflict-prone workloads (e.g., [anagram](#), [go.alpha](#)), enabling a small VC (2–4 entries) reduces the **effective miss rate** despite the same L1 miss rate, confirming that many L1 misses are converted into VC hits.
- Streaming or low-conflict tests see modest changes, as expected.
- The implementation of victim cache (VC) significantly improved cache performance by reducing the effective miss rate across multiple benchmarks. For instance, in the

anagram benchmark, the effective miss rate decreased by **42.6%** with 2 VC entries and **43.4%** with 4 entries. Similarly, in **test-math**, the improvement was **31.3%** at 2 entries and **33.2%** at 4 entries. The **test-fmath** benchmark saw the most notable gains, with **43.7%** improvement at 2 entries and **47.8%** at 4 entries. The **go.alpha** benchmark showed more modest improvements, with **6.3%** at 2 entries and **11.4%** at 4 entries. These results highlight the effectiveness of victim cache in reducing cache misses, particularly in computation-heavy workloads, with overall improvements ranging from **6.3% to 47.8%**.

Graph



5.2 Qualitative effects

- **Swap path correctness:** Verified that swapping preserves dirty state and recency.
- **Write-backs:** VC evictions of dirty lines trigger correct write-backs to L2.
- **Latency model:** With DL2/IL2 at 6 cycles and VC at 1 cycle, VC hits provide tangible AMAT improvement.

6. Baseline vs VC-Enabled File: Code-Level Comparison (Summary)

Below is a structured list of changes you will see when diffing the two `sim-cache.c` files:

1. Globals & Options

- `victim_cache` pointer added.
- `victim_entries` integer added.
- Registration of `-victim:entries` using option DB helpers.

2. Initialization

- VC construction conditional on `victim_entries > 0` using `cache_create("victim_%d", ...)` with 1 set and associativity = `victim_entries`.
- Adjusted DL2/IL2 hit latencies to 6 cycles.

3. Access Logic

- On DL1 miss: VC probe; on VC hit, **swap** line with L1 victim way; otherwise access DL2 as baseline.
- On DL1 eviction: insert evicted line into VC; on VC overflow, evict per LRU and write-back if dirty.

4. Statistics

- Added counters: `vc.hits`, `vc.misses`, `vc.evictions`, `vc.writebacks` and registration via `cache_reg_stats`.

5. Naming/Parsing

- VC name uses underscore to avoid issues in derived stat expressions.
-

7. Discussion

A small fully-associative VC effectively captures conflict-evicted lines, turning a notable share of L1 misses into near-hits. The degree of benefit depends on the workload's conflict behavior: access patterns mapping multiple hot lines to a small number of L1 sets gain the most. VC size beyond 4–8 lines often shows diminishing returns.

8. Conclusion

We verified the correctness of the VC integration and measured consistent reductions in **effective miss rate** across conflict-sensitive workloads. The implementation is compact, minimally invasive to the baseline, and yields clear gains with just a few entries, and with cache performance improvements ranging from **6.3% to 47.8%** across different benchmarks, **and its better to have victim cache than the Fully associative cache of bigger size because, in practical implementation larger cache sizes are slow and uses more power, so having victim cache is a better option for faster accesses.**

Appendix A: Run Commands

Baseline (no VC)

```
./sim-cache -victim:entries 0 -cache:dl1 dl1:128:32:1:l -cache:dl2 ul2:1024:64:4:l  
../Labs/Pisa/anagram
```

VC enabled (e.g., 4 entries)

```
./sim-cache -victim:entries 4 -cache:dl1 dl1:128:32:1:l -cache:dl2 ul2:1024:64:4:l  
../benchmarks/go.alpha 50 9 <../benchmarks/2stone9.in>
```

Appendix B: Effective Miss-Rate Formulae

- VC disabled: $Effective = L1_misses / L1_accesses$
- VC enabled: $Effective = VC_misses / L1_accesses$

References:

1) High Performance Cache Architecture Using Victim Cache -
<https://rjwave.org/ijedr/papers/IJEDR1703068.pdf>